

箕面ハイキングマップ 公開API 仕様書（契約バージョン 2.0）

バージョン: 1.0

最終更新日: 2026年8月18日

対象: minoh-hiking `api/` — 実装側 / MapPublisher — 呼び出し側

関連:

機能仕様 `funcspec-202608.md` §10（契約 1.0 の §10.2 は本書で置き換えた） /

MapPublisher `docs/migration-plan-202608.md`（移行の実装計画）

1. 本書について

1.1 目的

本書は、ハイキングマップアプリが表示するデータを外部の運用アプリから公開し、利用者アプリへ配信するための公開API の契約と、配信される GeoJSON の形式（公開スキーマ）を定義する。

1.2 正本

本書が正本である。API の実装（`api/`）と同じリポジトリに置く。

- 呼び出し側（MapPublisher）は本書に依存し、検証ルールを再実装しない
- 検証は API に任せ、失敗時は API が返した日本語メッセージをそのまま表示する
- 契約と公開スキーマを1冊にまとめるのは、検証ルールとスキーマがずれることを防ぐため

破壊的変更のときだけ契約バージョンを上げ、呼び出し側にも反映する。

1.3 契約 1.0 からの変更点

項目	契約 1.0	契約 2.0	理由
データセット	<code>closures</code> のみ	<code>mapdata</code> を追加（緊急ポイント・ルート・スポット）	アプリ同梱の geojson を公開経由の更新に変更する
受け付ける geometry	<code>Point</code> のみ	<code>Point</code> と <code>LineString</code>	ルートが <code>LineString</code> のため
<code>version</code>	クライアントが body に入れて送る（必須）	サーバーが採番する（送られても無視する）	上げ忘れによる「公開したのに届かない」を防ぐ
更新の通知	なし（毎回フル取得）	<code>GET /api/manifest</code> で version のみ先読み	<code>mapdata</code> が約 200KB あり、毎起動のフル取得を避ける
履歴	<code>history/</code> に無制限に追加	前回分1世代のみ（上書き）	保存量が公開回数に比例して増え続けるため
公開トークン	<code>CLOSURES_PUBLISH_TOKEN</code>	<code>MAP_PUBLISH_TOKEN</code> （2データセット共通）	対象が <code>closures</code> だけではなくなるため

2. データセット

2.1 一覧

データセット	日本語名	内容	version 形式	想定更新頻度
mapdata	ハイキングマップデータ	緊急ポイント・ハイキングルート・スポット	yyyy.n	年に数回
closures	通行止め・通行困難地点	通行止め・通行困難の地点	yyyy-mm.n	随時

日本語名は利用者向けの画面・文書で用いる呼称であり、**本書を正本とする**。

2.2 責務分担

主体	責務
MapEditor	地点・ルート・スポットの編集、通行止め地点の登録。作業用 GeoJSON を出力する
MapPublisher	作業用 GeoJSON を公開スキーマへ整形し、公開API へ送信する
公開API（本書）	検証・version 採番・保存・配信
minoh-hiking アプリ	配信データの表示。 GET のみ 利用する

編集用の識別子（spot の id など）は MapEditor の作業ファイルには残し、**MapPublisher が公開時に落とす**。編集の都合と配信の都合を分離する。

3. 公開スキーマ

配信される GeoJSON の形式を定義する。**利用者の表示に必要なものだけを出す**。

3.1 FeatureCollection

```
{
  "type": "FeatureCollection",
  "version": "2026.1",
  "updatedAt": "2026-08-18T05:12:34.567Z",
  "features": [ ... ]
}
```

キー	型	必須	設定者
type	"FeatureCollection"	○	送信側
version	string	○	サーバー（\$4）

キー	型	必須	設定者
<code>updatedAt</code>	string (ISO 8601)	○	サーバー (公開時刻)
<code>features</code>	array	○	送信側

`version` と `updatedAt` は送信側が入れてもサーバーの値で上書きされる。

3.2 mapdata の Feature

3.2.1 緊急ポイント (`ポイントGPS`)

プロパティ	型	必須	説明
<code>type</code>	" <code>ポイントGPS</code> "	○	固定値
<code>id</code>	string	○	例 " <code>B-01</code> "。現地の標識と対応する利用者向けの識別子のため保持する
<code>name</code>	string	○	例 " <code>聖天展望台</code> "
<code>description</code>	string	—	値があるときのみ出力する (null は出力しない)

geometry: `Point`

`pointId` は出力しない。全件で `id` と同値のため。

3.2.2 スポット (`spot`)

プロパティ	型	必須	説明
<code>type</code>	" <code>spot</code> "	○	固定値
<code>name</code>	string	○	例 " <code>滝道32鉄橋</code> "

geometry: `Point`

`id` / `source` / `description` は出力しない。

- `id` はどこからも参照されていない (ルートはスポットを `name` で参照する)。表示に必要なのは `name` と座標のみ
- `source` は全件が "`image_transformed`"、`description` は全件が "`スポット (GPS変換済)`" の定数

スポット名は一意ではない (「トイレ」「WC」など)。名称の重複は許容し、識別は座標で行う。

3.2.3 ルート (`route`)

プロパティ	型	必須	説明
<code>type</code>	" <code>route</code> "	○	固定値

プロパティ	型	必須	説明
id	string	○	例 "route_H-04_to_H-11"。区間の識別に使う
startPointGPS	[経度, 緯度, 標高?] null	○	開始点の座標
endPointGPS	[経度, 緯度, 標高?] null	○	終了点の座標

geometry: `LineString`（**中間点のみ**。開始点・終了点の座標は含まない）

`startPoint` / `endPoint`（ID参照）は**出力しない**。
端点の座標が入っているため ID による解決は不要であり、区間の識別は `id` で足りる。

利用者アプリは `startPointGPS` を `LineString` の先頭に、`endPointGPS` を末尾に補って描画する。
`null` の場合は補わない（その端は中間点から始まる）。

3.3 closures の Feature（`closure`）

プロパティ	型	必須	説明
type	"closure"	○	固定値
id	string	○	全地点で一意
name	string	○	地点名
kind	"closed" "difficult"	○	通行止め / 通行困難
reason	string	—	工事・倒木・落石 など。値があるときのみ
note	string	—	備考。値があるときのみ
relatedRoute	string	—	値があるときのみ
updatedAt	string	○	地点ごとの更新日時

geometry: `Point`

3.4 座標

- 形式: [経度, 緯度] または [経度, 緯度, 標高]
- 測地系: WGS84
- 小数点以下**5桁**に丸める（約1m精度）
- 標高の単位: メートル

3.5 出力例（mapdata）

```
{
  "type": "FeatureCollection",
  "version": "2026.1",
  "updatedAt": "2026-08-18T05:12:34.567Z",
  "features": [
    {
      "type": "Feature",
      "properties": { "type": "ポイントGPS", "id": "B-01", "name": "聖天展望台" },
      "geometry": { "type": "Point", "coordinates": [135.47258, 34.839, 184.6] }
    },
    {
      "type": "Feature",
      "properties": { "type": "spot", "name": "滝道32鉄橋" },
      "geometry": { "type": "Point", "coordinates": [135.47102, 34.84955, 162.3] }
    },
    {
      "type": "Feature",
      "properties": {
        "type": "route",
        "id": "route_H-04_to_H-11",
        "startPointGPS": [135.4713, 34.87451, 534],
        "endPointGPS": [135.47341, 34.86829, 552.9]
      },
      "geometry": {
        "type": "LineString",
        "coordinates": [[135.47142, 34.87301], [135.47205, 34.87088]]
      }
    }
  ]
}
```

4. version の採番

4.1 形式

データセット	形式	例
mapdata	yyyy.n	2026.1
closures	yyyy-mm.n	2026-08.1

n は1桁を前提とし、**ゼロ埋めしない**。

4.2 採番規則

1. `manifest.json` から現在の version を読み、期間（`yyyy / yyyy-mm`）と `n` に分解する
2. **サーバー時刻を JST（Asia/Tokyo）に変換**して現在の期間を求める
 - Vercel Functions は UTC で動作する。UTC のまま判定すると
9月1日 08:00 JST の公開が 8月扱いになるため、必ず変換すること
3. 期間が変わっていれば `n = 1`、同じであれば `n = 現在値 + 1`
4. 現在の version の期間がサーバー時刻より**未来**の場合（時計ずれ）は、
期間を戻さず `n` だけ加算する
5. version がパースできない場合（契約1.0 時代の手入力形式など）は、
現在の期間で `n = 1` から開始する

4.3 比較は等値のみ

`n` をゼロ埋めしないため、文字列の大小比較では `2026.10 < 2026.9` と逆転する。

- **更新判定は等値比較（`!==`）のみで行い、大小比較を実装しない**
- `n` が 10 に達した場合は2桁で継続する。動作に影響はなく、影響は表示順のみ

5. エンドポイント

5.1 一覧

メソッド	パス	認証	用途
GET	<code>/api/manifest</code>	不要	全データセットの version・件数（数百バイト）
GET	<code>/api/mapdata</code>	不要	mapdata の GeoJSON
POST	<code>/api/mapdata</code>	要	mapdata の公開（全置換）
GET	<code>/api/closures</code>	不要	closures の GeoJSON
POST	<code>/api/closures</code>	要	closures の公開（全置換）
OPTIONS	全て	不要	CORS プリフライト（204）

上記以外のメソッドは `405 Method Not Allowed`。

5.2 GET /api/manifest

```
{
  "mapdata": { "version": "2026.1",    "updatedAt": "2026-08-18T05:12:34.567Z",
  "count": 668 },
  "closures": { "version": "2026-08.1", "updatedAt": "2026-08-17T22:03:11.004Z",
  "count": 12  }
}
```

- `Cache-Control: no-store`
- `manifest.json` が未作成・取得失敗のときは、各データセットの本体から復元して返す
- それも取得できないデータセットは `{ "version": "", "updatedAt": null, "count": 0 }` を返す

5.3 GET /api/mapdata, GET /api/closures

- `Content-Type: application/geo+json; charset=utf-8`
- `Cache-Control: no-store`（キャッシュは端末側が担う）
- Blob 未作成・取得失敗時も **200 で空を返す**
(`{ "type": "FeatureCollection", "version": "", "features": [] }`)。アプリの表示を止めないため

5.4 POST /api/mapdata, POST /api/closures

- リクエスト: `Content-Type: application/json`、ヘッダ `x-publish-token`
- ボディ: 公開スキーマ (§3) の FeatureCollection
- **保存は全置換**（0件を送ると全消去になる）
- `version` / `updatedAt` は送られても無視し、サーバーの値を採用する

成功応答（200）：

```
{ "ok": true, "dataset": "mapdata", "version": "2026.2", "count": 669,
  "updatedAt": "2026-08-18T05:12:34.567Z" }
```

5.5 エラー応答

ステータス	意味	<code>error</code> の内容	呼び出し側の扱い
400	検証エラー	日本語の説明	データを直して再実行
401	トークン不正	固定文言	保存済みトークンを消して再入力
405	未対応メソッド	固定文言	—
500	保存失敗	日本語の説明	時間をおいて再実行 (§7.2 のとおり幂等)
503	サーバー側トークン未設定	固定文言	操作では直らない。設定が必要

呼び出し側は `error` の文言を**そのまま表示する**。文言を独自に持たない。

6. 検証ルール

問題がなければ受理し、あればエラーメッセージを返す。判定はここにのみ置く。

6.1 共通

1. `FeatureCollection` 形式であり `features` が配列であること
2. 各要素が `Feature` 形式で `geometry` を持つこと
3. 座標が箕面エリアの範囲内であること
 - 経度 `135.2` ~ `135.8` / 緯度 `34.6` ~ `35.1`
 - `LineString` は全頂点を検査する
 - `startPointGPS` / `endPointGPS` も同様に検査する (`null` は許容)
4. `id` が一意であること
 - `id` が未設定の `Feature` はスキップする。 `spot` (§3.2.2) は `id` を持たないため

6.2 mapdata 固有

- `properties.type` が `ポイントGPS` / `route` / `spot` のいずれかであること
- `geometry` は `Point` または `LineString` であること
- `ポイントGPS` / `spot` は `Point`
- `route` は `LineString`

6.3 closures 固有

- `geometry` は `Point` のみ

7. 保存仕様

7.1 Blob 構成

<code>manifest.json</code>	← 採番の基準・GET /api/manifest の実体
<code>mapdata/minoh-hiking-mapdata.geojson</code>	← 現行
<code>mapdata/previous.geojson</code>	← 前回分 (1世代のみ・毎回上書き)
<code>closures/minoh-hiking-closure.geojson</code>	← 現行
<code>closures/previous.geojson</code>	← 前回分 (1世代のみ・毎回上書き)

7.2 POST の書き込み順

1. `manifest.json` を読み、採番する (§4)
2. 検証する (§6)
3. 現行の本体を `previous` へ退避する

4. 本体を put する
5. `manifest.json` を put する
6. 200 を返す

手順5で失敗した場合は 500 を返す。`manifest.json` が進んでいないため、再実行すると同じ **version** が再採番される（冪等）。運用者は「もう一度公開」で復旧できる。

手順3で失敗しても**公開は成立させる**（警告ログのみ）。退避の失敗で公開を止めるほうが、運用上の害が大きい。手順3が成功して手順4が失敗した場合、`previous` が現行と同一内容になるだけで害はない。

7.3 前回分の保持

```
// @vercel/blob の copy()。本体をダウンロードせずに退避できる
await copy(BLOB_PATH, PREVIOUS_PATH, {
  access: 'public',
  contentType: 'application/geo+json', // copy は metadata を引き継がないため再指定
  する
  addRandomSuffix: false,
  allowOverwrite: true
});
```

初回公開時は本体が存在せず `BlobNotFoundError` になる。握りつぶして続行する。

8. 認証

- 環境変数 `MAP_PUBLISH_TOKEN`（Vercel に設定。コミット禁止）
- ヘッダ `x-publish-token`
- 比較は**固定長ハッシュ同士**で行う（SHA-256 → `timingSafeEqual`）。タイミング攻撃対策
- 環境変数が未設定のときは 503 を返す
- 2つのデータセットで**同一のトークン**を使う

9. CORS

- `Access-Control-Allow-Origin: *`
- `Access-Control-Allow-Methods: GET, POST, OPTIONS`
- `Access-Control-Allow-Headers: Content-Type, x-publish-token`

GET は公開データであり、POST はトークンで保護されるため Origin を限定しない。
GitHub Pages 版アプリ・MapPublisher からもクロスオリジンで参照するため許可が必要。
呼び出し側に **CORS 対応の追加実装は不要**。

10. 利用側アプリの更新判定

利用者アプリ（minoh-hiking）は **GET のみ** を使う。

1. localStorage から保存済み version を読む

2. Cache API から GeoJSON を読んで即描画

3. GET /api/manifest (cache: 'no-store')

└ 失敗（オフライン等）

└ version が一致

└ version が相違

4. GET 本体

5. Cache API に保存

6. 再描画

7. localStorage の version を更新

← オフラインでもここまでで表示される

→ 終了

→ 終了（本体を取りに行かない）

← ★必ず最後

手順7を先に行ってはならない。本体の取得・保存に失敗したあとに version だけが進むと、以後その端末は永久に更新されなくなる。

一度もオンラインで起動していない端末は表示なしとなる。
その状態では地図タイルも未取得のため、運用上の問題としない。

11. 実装しないこと

意図的に持たない。将来「不足している」と見えても、以下の理由で追加しない。

項目	理由
誤公開に対する件数下限チェック	公開前の確認は呼び出し側のダイアログで行う。サーバー側に閾値判定を持つと、正当な一括削除ができなくなる
呼び出し側での検証の追加	判定は本書の §6 にのみ置く。二重管理は必ずずれる
本書の内容の呼び出し側への転記	同上。呼び出し側は 依存している項目だけ を列挙し、ルールは書き写さない
公開前の実機プレビュー	「公開 → アプリで確認 → 必要なら再公開」の運用とする。全置換＋前回分の保持があるため復旧できる
データセットごとのトークン分離	運用者は1名。共通トークンで足りる

12. 変更履歴

版	日付	内容
1.0	2026-08-18	初版。契約バージョン 2.0 を定義し、機能仕様書 §10.2（契約 1.0）を置き換える